



Terraform Day-2

Providers, Variables, Functions and AWS VPC + EC2 Project

AWS

Prepared by Satyanarayan Sen

8

pages of visual Day-2 study notes

9

repo lessons compressed into one flow

1

VPC + EC2 + ALB project walkthrough

0

real credentials or live AWS actions

1 DAY-2 GOAL

Move from basic Terraform commands to reusable, provider-aware code.

- Connect Terraform to AWS safely.
- Use variables and outputs.
- Prepare for multi-region projects.
- Build a small public web stack.

2 LEARNING MAP

1. Providers
2. Required providers
3. Multiple providers
4. Provider aliases
5. Variables and outputs
6. .tfvars files
7. Conditionals
8. Built-in functions
9. VPC with EC2 project

3 PREREQUISITES

- Terraform CLI installed.
- AWS CLI installed.
- AWS account for practice only.
- VS Code or any code editor.
- Day-1 basics: init, plan, apply, destroy.

IaC

HCL

AWS

Networking

4 TERRAFORM THINK MODEL



Providers translate Terraform configuration into cloud API calls. Variables make the same code reusable across environments.

5 SAFETY PROMISE

- Do not paste access keys into Terraform files.
- Use least-privilege IAM users or roles.
- Prefer small practice accounts and clean up resources.
- Review plan output before every apply.

This note explains the project. It does not run Terraform or deploy AWS resources.

Provider

Plugin that talks to an API such as AWS, Azure, Google Cloud, Kubernetes, or VMware.

Variable

Input value that makes Terraform code configurable instead of hardcoded.

Output

Useful value Terraform prints after creating or reading infrastructure.

Function

Helper expression used to transform lists, maps, strings, and numbers.

Source: iam-veeramalla/terraform-zero-to-hero Day-2

Beginner-safe edition

Source reference: <https://github.com/iam-veeramalla/terraform-zero-to-hero/tree/main/Day-2>. Modern Terraform notes are aligned with current HashiCorp function guidance.

DAY 2



Providers

How Terraform connects to cloud platforms and APIs

Prepared by Satyanarayan Sen

1 WHAT IS A PROVIDER?

A provider is a Terraform plugin that knows how to call one platform API.

- `aws` creates AWS resources.
- `azurerm` creates Azure resources.
- `google` creates Google Cloud resources.
- `kubernetes` manages Kubernetes objects.

2 SIMPLE ANALOGY

Terraform is the project manager. The provider is the specialist contractor for one platform.



3 WHEN IT RUNS

- `terraform init` downloads providers.
- `terraform plan` asks providers what will change.
- `terraform apply` asks providers to create or update resources.
- `terraform destroy` asks providers to remove managed resources.

4 AWS PROVIDER EXAMPLE

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "example" {
  ami           = "ami-0123456789abcdef0"
  instance_type = "t2.micro"
}
```

The provider block tells Terraform which AWS region to use. The resource block describes the EC2 instance.

5 REQUIRED PROVIDERS BLOCK

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

This pins where the provider comes from and what version range Terraform should install.

6 PROVIDER SOURCE

`hashicorp/aws` means the official AWS provider from the Terraform Registry namespace `hashicorp`.

7 VERSION CONSTRAINT

`~> 5.0` allows compatible 5.x versions. Pinning avoids surprise upgrades in production.

8 DO NOT STORE KEYS

Use environment variables, AWS profiles, SSO, or IAM roles. Never commit access keys into `.tf` files.

9 PROVIDER SETUP CHECKLIST



DAY 2



Multiple Providers

One Terraform project can work with more than one platform or region

Prepared by Satyanarayan Sen

1 MULTI-CLOUD IDEA

A single root module can configure AWS and Azure providers, then create resources in each platform.

```
provider "aws" {
  region = "us-east-1"
}

provider "azurerm" {
  features {}
}

resource "aws_instance" "web" {
  ami           = "ami-0123456789abcdef0"
  instance_type = "t2.micro"
}
```

2 SAME PROVIDER, MULTIPLE REGIONS

Use provider aliases when one provider needs more than one configuration.

```
provider "aws" {
  alias = "east"
  region = "us-east-1"
}

provider "aws" {
  alias = "west"
  region = "us-west-2"
}

resource "aws_instance" "east_web" {
  provider = aws.east
  ami      = "ami-0123456789abcdef0"
  instance_type = "t2.micro"
}
```

3 WHY USE ALIASES?

- Deploy similar resources in two regions.
- Keep disaster recovery infrastructure ready.
- Split workloads by latency, compliance, or cost.

4 PROVIDER SELECTION

When a resource should use a non-default provider, set the `provider` argument.

```
resource "aws_s3_bucket" "logs" {
  provider = aws.west
  bucket   = "example-logs"
}
```

5 MODULE PATTERN

Pass provider aliases into child modules when reusable modules need a specific region.

```
module "network" {
  source = "../modules/network"
  providers = {
    aws = aws.east
  }
}
```

6 DECISION FLOW



7 COMMON MISTAKE

Writing `provider = "aws.us-east-1"` is legacy-style. Modern Terraform uses provider references like `provider = aws.east`.

8 COMMON MISTAKE

Using the same resource name twice in one module causes conflicts. Use unique local names.

9 COMMON MISTAKE

Forgetting region-specific AMI IDs. AMIs are usually region-specific, so check each region.

Rule: default provider for normal resources, alias providers for special locations.

Review every plan.

DAY 2



Variables, Outputs, tfvars

Make Terraform reusable, readable, and environment-aware

Prepared by Satyanarayan Sen

1 INPUT VARIABLES

Inputs let the same configuration accept different values.

```
variable "instance_type" {
  description = "EC2 size"
  type        = string
  default     = "t2.micro"
}
```

2 USE VARIABLES

Reference variables with the `var.` prefix.

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = var.instance_type
}
```

3 OUTPUT VALUES

Outputs expose useful information after Terraform finishes.

```
output "public_ip" {
  value = aws_instance.web.public_ip
}
```

4 .TFVARS FILES

A `.tfvars` file stores values for variables, keeping environment settings separate from reusable code.

```
# dev.tfvars
instance_type = "t2.micro"
ami_id        = "ami-0123456789abcdef0"
environment   = "dev"
```

Run with: `terraform plan -var-file=dev.tfvars`

5 ENVIRONMENT PATTERN

File	Purpose	Example
variables.tf	Questions	What inputs exist?
dev.tfvars	Dev answers	Small instance, test CIDR
stage.tfvars	Staging answers	Medium instance
prod.tfvars	Production answers	Approved production values

string

Single text value such as region or AMI ID.

number

Numeric value such as instance count.

list

Ordered collection such as subnet CIDRs.

map

Key-value collection such as tags.

6 VARIABLE IMPLEMENTATION DEMO

```
variable "cidr" {
  default = "10.0.0.0/16"
}

resource "aws_vpc" "myvpc" {
  cidr_block = var.cidr
}
```

This mirrors the Day-2 project: the VPC CIDR is controlled from `variables.tf`.

7 SECRETS WARNING

- Do not commit `.tfvars` files that contain passwords or keys.
- Add secret tfvars files to `.gitignore`.
- Use secret managers, environment variables, or CI/CD protected variables.
- Keep example files such as `dev.tfvars.example` safe and fake.

Remember: variables ask questions, tfvars answer them, outputs show results.

Reusable Terraform

DAY 2



Conditionals and Functions

Make decisions and transform values inside HCL

Prepared by Satyanarayan Sen

1 TERNARY SYNTAX

```
condition ? true_value : false_value
```

Terraform returns the first value when the condition is true, otherwise the second value.

2 CONDITIONAL RESOURCE

```
resource "aws_instance" "example" {  
  count = var.create_instance ? 1 : 0  
  ami   = var.ami_id  
}
```

3 CONDITIONAL CONFIG

```
cidr_blocks = var.enable_ssh  
  ? ["0.0.0.0/0"]  
  : []
```

4 ENVIRONMENT EXAMPLE

```
variable "environment" {  
  type    = string  
  default = "development"  
}  
  
locals {  
  subnet_cidr = var.environment == "production"  
    ? "10.0.1.0/24"  
    : "10.0.2.0/24"  
}
```

Use conditionals for small choices. If logic becomes large, redesign with clearer variables or maps.

5 SECURITY REMINDER

Conditionals can accidentally open access. Review values before apply, especially security group CIDRs and `count` values.

- Production SSH should not be `0.0.0.0/0`.
- Empty lists may remove access.
- `count = 0` destroys managed instances during apply.

concat()

Combines lists.

```
concat(["a"], ["b"])
```

element()

Selects a list item by index.

```
element(["a", "b"], 1)
```

length()

Counts items in a list, map, or string.

```
length(var.subnets)
```

lookup()

Reads a map key with a fallback.

```
lookup(var.tags, "env", "dev")
```

join()

Turns a list into one string.

```
join(" ", var.names)
```

zipmap()

Creates a map from keys and values.

```
zipmap(var.keys, var.values)
```

6 MODERN FUNCTION NOTE

The repo demonstrates the old `map()` function idea. Current Terraform removed `map()`. Use map literals, `tomap()`, or `zipmap()` instead.

```
locals {  
  tags = tomap({  
    Name = "web"  
    Env  = "dev"  
  })  
}
```

7 FUNCTION MINDSET

Need	Use
Merge list values	`concat()`
Get one list item	`element()`
Count values	`length()`
Read map safely	`lookup()`
Build map from lists	`zipmap()`

DAY 2

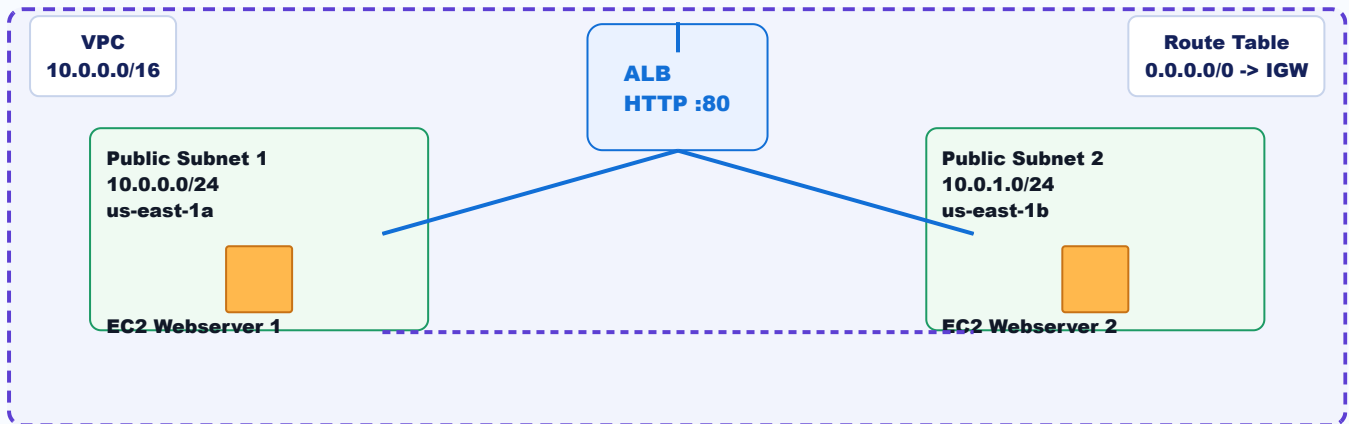


Project Architecture

AWS VPC with two EC2 web servers behind an Application Load Balancer

Prepared by Satyanarayan Sen

Internet Gateway + Public Internet



VPC: private network boundary

Subnets: public AZ placement

ALB: traffic distribution

Output: load balancer DNS

1 NETWORK LAYER

- `aws\_vpc.myvpc` with `var.cidr`.
- Two public subnets in two AZs.
- Internet gateway attached to VPC.
- Route table sends internet traffic to IGW.

2 COMPUTE LAYER

- Two EC2 instances.
- Each instance uses Apache user data.
- Security group allows HTTP and SSH.
- Instances register into ALB target group.

3 TRAFFIC FLOW

1. User opens ALB DNS.
2. ALB receives HTTP traffic.
3. Listener forwards to target group.
4. Target group sends traffic to healthy EC2.

VPC

Own logical network in AWS.

IGW

Gateway between VPC and internet.

SG

Instance and ALB firewall rules.

ALB

HTTP entry point across two servers.

Beginner-safe fix: the repo bucket name is hardcoded and globally unique to the original author. In practice, use your own unique bucket name variable or remove the bucket if the demo does not need it.

DAY 2



Project Build Walkthrough

Important files and commands from Day-2 PROJECT-vpc-with-ec2

Prepared by Satyanarayan Sen

1 FOLDER STRUCTURE

```
PROJECT-vpc-with-ec2/  
|-- provider.tf  
|-- variables.tf  
|-- main.tf  
|-- userdata.sh  
|-- userdata1.sh
```

2 PROVIDER.TF

```
terraform {  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "5.11.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "us-east-1"  
}
```

3 VARIABLES.TF

```
variable "cidr" {  
  default = "10.0.0.0/16"  
}
```

The VPC CIDR is kept outside `main.tf` so it can be changed safely.

4 MAIN.TF RESOURCE ORDER

1. Create VPC.
2. Create two public subnets.
3. Create internet gateway.
4. Create route table and associations.
5. Create web security group.
6. Create EC2 web servers.
7. Create ALB, target group, attachments, listener.
8. Output ALB DNS name.

5 CORE SNIPPET

```
resource "aws_vpc" "myvpc" {  
  cidr_block = var.cidr  
}  
  
resource "aws_subnet" "sub1" {  
  vpc_id            = aws_vpc.myvpc.id  
  cidr_block        = "10.0.0.0/24"  
  availability_zone = "us-east-1a"  
  map_public_ip_on_launch = true  
}
```

6 USER DATA

The scripts install Apache, create `index.html`, show the instance ID, then start and enable the web server.

```
user_data = base64encode(  
  file("userdata.sh")  
)
```

7 ALB OUTPUT

```
output "loadbalancerdns" {  
  value = aws_lb.myalb.dns_name  
}
```

After apply, this DNS name is the browser entry point.

8 PRACTICE CAUTION

The security group opens HTTP and SSH to the world in the repo demo. For real systems, restrict SSH to trusted IP ranges.

9 COMMAND FLOW

terraform
init



terraform
validate



terraform
plan



terraform
apply



terraform
destroy

Always run destroy after short AWS practice labs to avoid surprise charges.

Cost control

DAY 2



Validation and Interview Prep

Check your work, clean up AWS, and explain Day-2 confidently

Prepared by Satyanarayan Sen

1 VALIDATION CHECKLIST

- `terraform fmt -check` for formatting.
- `terraform validate` for syntax and provider schema checks.
- `terraform plan` for review before create/update.
- Confirm ALB DNS output after apply.
- Check EC2 targets are healthy in ALB target group.

2 CLEANUP CHECKLIST

- Run `terraform destroy` for labs.
- Confirm EC2, ALB, target groups, and NAT-like paid items are gone.
- Check S3 buckets manually if destroy fails.
- Review AWS billing dashboard.

3 RISK NOTES

- Open SSH to world is unsafe.
- ALB and EC2 can create costs.
- Public subnets expose instances directly.
- Hardcoded AMIs can become outdated.

4 INTERVIEW QUESTIONS

1. What is a Terraform provider?
2. Why use `required\_providers`?
3. How do provider aliases help in multi-region deployments?
4. What is the difference between variables and tfvars?
5. When should you use outputs?
6. What does `count = condition ? 1 : 0` do?
7. Why is remote state useful in teams?
8. What does an ALB do in the Day-2 project?

5 QUICK ANSWERS

Concept	One-line answer
Provider	Plugin that lets Terraform call a platform API.
Variable	Input that makes code configurable.
tfvars	File that supplies variable values.
Output	Value printed after Terraform evaluates infrastructure.
Alias	Named provider configuration for another region or account.
ALB	HTTP load balancer that forwards traffic to healthy targets.

Production Add-on

Use remote backend in S3 with DynamoDB locking.

Production Add-on

Move EC2 into private subnets behind ALB.

Production Add-on

Use modules for VPC, EC2, and ALB.

Production Add-on

Add HTTPS using ACM and Route 53.

6 DAY-2 CHEAT SHEET

```
terraform init
terraform fmt -check
terraform validate
terraform plan -var-file=dev.tfvars
terraform apply -var-file=dev.tfvars
terraform destroy -var-file=dev.tfvars
```

7 RESUME PROJECT LINE

AWS VPC Web Application Infrastructure using Terraform

Automated VPC, public subnets, internet gateway, route table, EC2 web servers, security group, Application Load Balancer, target group, listener, and DNS output using Terraform.

Day-2 Outcome: reusable Terraform code + provider control + deployable AWS web architecture.

Happy Learning

Sources: GitHub Day-2 repo; HashiCorp Terraform language docs for functions and provider behavior. Notes adapted for beginner safety and current Terraform usage.

DAY 2